Tarrataca's Quantum Production System - 3-Puzzle

Quantum AI (417523A) - SPPU BE AI&DS

CODE

```
import math
import warnings
from collections import deque

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

warnings.filterwarnings("ignore", category=DeprecationWarning)

NEIGHBOURS = {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}
START = (0, 1, 2, 3)
GOAL = (3, 2, 1, 0)


def draw(state):
    cell = [" " if t == 0 else str(t) for t in state]
    return "[ %s %s ] [ %s %s ]" % tuple(cell)


def successors(state):
    i = state.index(0)
    out = []
    for j in NEIGHBOURS[i]:
        nxt = list(state)
        nxt[i], nxt[j] = nxt[j], nxt[i]
        out.append(tuple(nxt))
    return out


def state_space(start):
    seen, queue, order = {start}, deque([start]), [start]
    while queue:
        for nxt in successors(queue.popleft()):
            if nxt not in seen:
                seen.add(nxt)
                order.append(nxt)
                queue.append(nxt)
    return order


def phase_flip(target, n, qc):
    for q in range(n):
        if not (target >> q) & 1:
            qc.x(q)
    qc.h(n - 1)
    qc.mcx(list(range(n - 1)), n - 1)
    qc.h(n - 1)
    for q in range(n):
        if not (target >> q) & 1:
            qc.x(q)


def diffuser(n, qc):
    qc.h(range(n))
    qc.x(range(n))
    qc.h(n - 1)
    qc.mcx(list(range(n - 1)), n - 1)
    qc.h(n - 1)
    qc.x(range(n))
    qc.h(range(n))


print("=" * 66)
print("  QUANTUM AI (417523A)  -  PRACTICAL 3")
```

```
print("  TARRATACA'S QUANTUM PRODUCTION SYSTEM  -  3-PUZZLE")
print("=" * 66)

states = state_space(START)
index_of = {s: i for i, s in enumerate(states)}
goal_index = index_of[GOAL]

print(f"\nSTART state : {draw(START)}  {START}")
print(f"GOAL  state : {draw(GOAL)}  {GOAL}")
print("\nProduction rules applied to START (the blank moves to a neighbour):")
for rule, nxt in enumerate(successors(START), 1):
    print(f"   rule {rule}: {draw(nxt)}  =  {nxt}")

n = math.ceil(math.log2(len(states)))
N = 2 ** n
iterations = math.floor(math.pi / 4 * math.sqrt(N))

print(f"\nState space enumerated by breadth-first search : {len(states)} states")
print(f"Index of the GOAL state in that enumeration    : {goal_index}")
print(f"Grover search register                         : {n} qubits  (N = {N})")
print(f"Iterations floor(pi/4 * sqrt(N))               : {iterations}")

qc = QuantumCircuit(n, n)
qc.h(range(n))
for _ in range(iterations):
    phase_flip(goal_index, n, qc)
    diffuser(n, qc)
qc.measure(range(n), range(n))

sim = AerSimulator()
counts = sim.run(transpile(qc, sim), shots=4096).result().get_counts()
ranked = sorted(counts.items(), key=lambda kv: -kv[1])

print("\nGrover measurement outcomes (top 5 of 4096 shots):")
for bits, count in ranked[:5]:
    print(f"   |{bits}> = index {int(bits, 2):2d} : {count:5d}  ({100 * count / 4096:5.2f} %)")

best_bits, best_count = ranked[0]
found = int(best_bits, 2)
found_state = states[found] if found < len(states) else None

print("\nMost probable index returned by the oracle :", found)
print("Matching bitstring                         :", best_bits)
print("Decoded puzzle state                       :", draw(found_state) if found_state else "not a legal
state")
print("Target GOAL state                          :", draw(GOAL))
print("Found state equals GOAL (classical check) :", found_state == GOAL)
print(f"Confidence                                  : {100 * best_count / 4096:5.2f} %")
print("=" * 66)
```

OUTPUT

```
==================================================================
  QUANTUM AI (417523A)  -  PRACTICAL 3
  TARRATACA'S QUANTUM PRODUCTION SYSTEM  -  3-PUZZLE
==================================================================

START state : [   1 ] [ 2 3 ]  (0, 1, 2, 3)
GOAL  state : [ 3 2 ] [ 1   ]  (3, 2, 1, 0)

Production rules applied to START (the blank moves to a neighbour):
   rule 1: [ 1   ] [ 2 3 ]  =  (1, 0, 2, 3)
   rule 2: [ 2 1 ] [   3 ]  =  (2, 1, 0, 3)

State space enumerated by breadth-first search : 12 states
Index of the GOAL state in that enumeration    : 11
Grover search register                         : 4 qubits  (N = 16)
Iterations floor(pi/4 * sqrt(N))               : 3

Grover measurement outcomes (top 5 of 4096 shots):
   |1011> = index 11 :  3953  (96.51 %)
   |0100> = index  4 :    13  ( 0.32 %)
   |0111> = index  7 :    12  ( 0.29 %)
```

```
   |1100> = index 12 :    12  ( 0.29 %)
   |1111> = index 15 :    11  ( 0.27 %)

Most probable index returned by the oracle : 11
Matching bitstring                         : 1011
Decoded puzzle state                       : [ 3 2 ] [ 1   ]
Target GOAL state                          : [ 3 2 ] [ 1   ]
Found state equals GOAL (classical check) : True
Confidence                                 : 96.51 %
======================================================================
```

FIGURE: Q3_grover_production_system

